SOREL CAN Bus Interface (SCBI) API Reference

1. Goals for Developing the SCBI

1. Provide read access to the controller's state, i.e. useful internal variables.
   Example use case: datalogger
2. Supply other CAN bus subscribers with information on preconfigured events.
   Examples:
   - Controller with ID=0x39 switched to “eco mode”
   - Controller with ID=0xFA requests additional heat
3. Be able to control other CAN bus devices

2. Technical Description

2.1 Overview

table 1: Structure of a SOREL CAN Package

Address ID 29 bits	bit 0..7: Program Type	Control program. Defines which actions can be performed and sets the context, in which they are performed.
	bit 8..15: Subscriber ID	unique CAN Subscriber ID of the sender
	bit 16..23: Function Type	action to perform Each Program Type provides a different set of actions.
	bit 24..26: Protocol Type	One for payloads <=8 Bytes and one for payloads >8 Bytes.
	bit 27..28: CAN Message Type	type of message Each Protocol Type has different Message Types.
Body 64 bits	bit 29..92: Payload	up to 8 Bytes of data per CAN package

Address ID
The SCBI is derived from the CAN-Open protocol. As the basis of the interface a 29 bit extended address resp. extended identifier (29bit Ext ID) is used. Address ID format for a specific “Protocol Type”:

Format: `ProgramType . SubscriberID . FunctionType . CANMessageType`
Example: `CONTROLLER . SubscriberID . I_AM_HERE . MSG_RESPONSE`

The Protocol Type is impliced and thus not mentionend. Remember to interpret the Address ID with Big Endian.

Protocol Type
All current projects use one of the following Protocol Types:

- **`(0h) CAN_FORMAT_0`** Transmit a single CAN package with a payload of *up to 8 Bytes*
- **`(1h) CAN_FORMAT_BULK`** Transmit multiple CAN packages for payloads *larger than 8 Bytes*
- **`(2h) CAN_FORMAT_UPDATE`** Transmit a firmware update

The protocol type is encoded within bits 24 to 26 of the Address ID. Depending on which “Protocol Type” and “Program Type” is selected , the meaning of the remaining bits changes, i.e. for each Program Type a certain Function Type value can have a different meaning and depending on the Protocol Type, CAN Message Type values have a different meaning as well.

Note: For certain telegrams this protocol uses RTR to transmit information solely via the Address ID thus saving bandwidth by not sending a payload. This is typically done for e.g. confirmations and some responses.
Note: BROADCAST_ID == 0

2.2 Protocol Type: (0) CAN_FORMAT_0

table 2: Structure of a CAN Package (Protocol Type: CAN_FORMAT_0)

Address ID 29 bits	bit 0..7: Program Type	0x0B CONTROLLER 0x83 REMOTESENSOR 0x84 DATALOGGER_NAMEDSENSORS = REMOTESENSOR (for compatibility) 0x85 HCC 0x8C AVAILABLERESOURCES 0x90 PARAMETERSYNCCONFIG 0x91 ROOMSYNC 0x94 MSGLOG
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	*action to perform*
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0**
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*data*

CAN_FORMAT_0 is used to transmit payloads smaller than 8 Byte.

CAN Message Types:

```
(0) MSG_REQUEST    ...
(1) MSG_RESERVE    ...
(2) MSG_RESPONSE   ...
(3) MSG_ERROR      ...
```

2.2.1 Program Type: (0x0B) CONTROLLER

table 3: Structure of a CAN Package (Program Type: CONTROLLER)

Address ID 29 bits	bit 0..7: Program Type	**(0x0B): CONTROLLER**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	0 HAS_ANYBODY_HERE = 0 // discover CAN subscribers 1 I_AM_HERE = 1 2 GET_CONTROLLER_ID 3 GET_ACTIVE_PROGRAMS_LIST 4 ADD_PROGRAM 5 REMOVE_PROGRAM 6 GET_SYSTEM_DATE_TIME 7 SET_SYSTEM_DATE_TIME 8 I_AM_RESETED 9 DATALOGGER_TEST = 9
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0**
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*see individual description below*

(0) HAS_ANYBODY_HERE
On the first start, when a controller's CAN Subscriber ID is still undefined, it sends this message:

```
* Address ID: CONTROLLER.BROADCAST_ID.HAS_ANYBODY_HERE.MSG_RESPONSE
* Body      : (0) RTR
```

Every controller listening on the CAN bus responds with an I_AM_HERE telegram containing their CAN Subscriber ID. This exchange allows the new controller to choose an unused Subscriber ID.

(1) I_AM_HERE
Every 60 seconds each controller sends the message I_AM_HERE to every other controller.

```
* Address ID: CONTROLLER.BROADCAST_ID.I_AM_HERE.MSG_RESPONSE
* Body      : SubscriberID.CFG_DEVICE_ID.CFG_OEM_ID.CFG_DEVICE_VARIANT
```

(8) I_AM_RESETED
This message is send, as soon as a controller completed a restart and already has a CAN ID.

```
* Address ID: CONTROLLER.BROADCAST_ID.I_AM_RESETED.MSG_RESPONSE
* Body      : SubscriberID.CFG_DEVICE_ID.CFG_OEM_ID.CFG_DEVICE_VARIANT
```

(9) DATALOGGER_TEST
A variety of messages needed to carry out the hardware test of the datalogger.

2.2.2 Program Type: (0x80) DATALOGGER_MONITOR

table 4: Structure of a CAN Package (Program Type: DATALOGGER_MONITOR)

Address ID 29 bits	bit 0..7: Program Type	**(0x80): DATALOGGER_MONITOR**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	0 DLF_UNDEFINE = 0 1 DLF_SENSOR 2 DLF_RELAY 3 DLG_HYDRAULIK_PROGRAMM 4 DLG_ERROR_MESSAGE 5 DLG_PARAM_MONITORING 6 DLG_STATISTIC 7 DLG_OVERVIEW 8 DLG_HYDRAULIK_CONFIG
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0**
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*see individual description below*

(3) DLG_HYDRAULIK_PROGRAMM
Request the number of the currently selected control program from a specific controller via its ContollerID.

```
* Address ID: DATALOGGER_MONITOR.SubscriberID.DLG_HYDRAULIK_PROGRAMM.MSG_REQUEST
* Body      : (0)
```

(3) DLG_HYDRAULIK_PROGRAMM
Response containing the control program number

```
* Address ID: DATALOGGER_MONITOR.SubscriberID.DLG_HYDRAULIK_PROGRAMM.MSG_RESPONSE
* Body      : [max. 8 characters]   the current program's name
```

(8) DLG_HYDRAULIK_CONFIG
Controller parameter: currently selected control program number, number of relays and sensors

```
* AddressID: DATALOGGER_MONITOR.SubscriberID.DLG_HYDRAULIK_CONFIG.MSG_RESPONSE
* Body     : v1: N_PHYSICAL_SENSORS, N_RELAIS_PARAMS, 0, 0
             v2: NabtoSensorCount, NabtoRelayCount, TempFraction, overviewEarningCountDL
```

TempFraction: 0x00 -> FALSE, 0xFF -> TRUE

(1) DLF_SENSOR

Contains the current sensor value

```
* Address ID: DATALOGGER_MONITOR.SubscriberID.DLF_SENSOR.MSG_RESPONSE
* Body      : SensorNo.Value
      - SensorNo  [1 Byte]
      - value     [32 bit integer] (message len = 5)
      - value     (1 Byte) optional sensor type (message len = 7)
      - value     (1 Byte) optional sensor sub type (message len = 7)

Currently used sensor types
0; unknown
1: flow (vfs sensor)
2: relPressure (rps sensor)
3: diffPressure (dps sensor)
4: temperature
5: humididy
6: room controller wheel
7: room controller switch

Sub types for vfs
0: off
1: 1-12l/min
2: 1-20l/min
3: 2-40l/min
4: 5-100l/min
5: 10-200l/min
6: 20-400l/min
7: DST_VVX15,
8: DST_VVX20,
9: DST_VVX25,

Sub types for rps
0: off
1: 0-0,6 bar
2: 0-1 bar
3: 0-1,6 bar
4: 0-2,5 bar
5: 0-4 bar
6: 0-6 bar
7: 0-10 bar
```

(2) DLF_RELAY

Contains the current relay value

```
* Address ID: DATALOGGER_MONITOR.SubscriberID.DLF_RELAY.MSG_RESPONSE
* Body:
      - RelayNo [1 Byte]

      - mode    [1 Byte]
            0  RELAISMODE_SWITCHED     // on/off
            1  RELAISMODE_PHASE        // phase angle speed control
            2  RELAISMODE_PWM
            3  RELAISMODE_VOLTAGE

      - value   [1 Byte]

      - ExFunction: extra function connected to the relay   [1 Byte]
           -2  EF_DISABLED    = -2
           -1  EF_UNSELECTED  = -1

            0  EF_SOLARBYPASS = 0
            1  EF_HEIZEN                  // Heizen == heating
            2  EF_HEIZEN2
            3  EF_KUEHLEN                 // Kuehlen == cooling
            4  EF_RUECKLAUFANHEBUNG       // Ruecklaufanhebung = return flow increase
            5  EF_DISSIPATION
            6  EF_ANTILEGIO
            7  EF_UMLADUNG                // Umladung == reverse loading
            8  EF_DIFFERENZ               // Differenz == difference
            9  EF_HOLZKESSEL              // Holzkessel == wood boiler

           10  EF_SCHUTZFUNKTION          // Schutzfunktion == safety function
           11  EF_DRUCKREGELUNG           // Druckregelung == pressure control
           12  EF_BOOSTER
           13  EF_R1PARALLELBETRIEB       // Parallelbetrieb == parallel operation
           14  EF_R2PARALLELBETRIEB
           15  EF_DAUEREIN                // dauer-ein == always on
           16  EF_HEIZKREISRC21           // Heizkreis == heating circuit
           17  EF_CIRCULATION
           18  EF_STORAGEHEATING
           19  EF_STORAGESTACKING

           20  EF_R_V1_PARALLEL
           21  EF_R_V2_PARALLEL
           22  EF_R1_PERMANENTLY_ON
           23  EF_R2_PERMANENTLY_ON
           24  EF_R3_PERMANENTLY_ON = 24

           25  EF_V2_PERMANENTLY_ON = 25
           26  EF_EXTERNALALHEATING
           27  EF_NEWLOGMESSAGE

           28  EF_EXTRAPUMP
           29  EF_PRIMARYMIXER_UP
           30  EF_PRIMARYMIXER_DOWN
           31  EF_SOLAR
           32  EF_CASCADE

      - ExFunction: extra function previously connected to the relay   [1 Byte]
           … same ExFunc values as above …
```

(3) DLG_HYDRAULIK_PROGRAMM

…

```
* Address ID: ...
* Body:
      - HydraulicProgramName  [maximum of 8 Bytes]
```

(4) DLG_ERROR_MESSAGE

Contains MessageLogEntry. Transmitted using CAN_FORMAT_BULK. One package in the response bulk has a maximum length of 8 Bytes. The first Byte contains the transaction index (uint8_t). The next 7 Bytes contain the error message as an ASCII string.

(5) DLG_PARAM_MONITORING

Complete parameter information (network name, value, min value, max value, …)
Transmitted via CAN_FORMAT_BULK.

(6) DLG_STATISTIC

…

```
* Address ID: ...
* Body:
      - index   [1 Byte]
             0  BETRIEB_ART = 0      // Betriebsart == operation mode
             1  LEISTUNG             // Leistung == power
             2  LEISTUNG_VFS1
             3  LEISTUNG_VFS2
             4  ERTRAG_CONST         // Ertrag == heat yield
             5  ERTRAG_VFS1
             6  ERTRAG_VFS2
             7  ERTRAG_TOTAL         // totaler Ertrag == total heat yield
             8  WMZ_AKTIVE
             9  VFS1
            10  VFS2

      - value   [32 bit integer]
```

(7) DLG_OVERVIEW

…

```
* Address ID: ...
* Body:
      - type   [bits 8 to 6 of byte[0]]
            1  DAYS = 1
            2  WEEKS
            3  MONTHS
            4  YEARS
            5  TOTAL
            6  STATUS

      - index       [bits 5 to 1 of byte[0]]
      - hours       [16 bit integer]
      - heat yield  [32 bit integer]
```

(8) DLG_HYDRAULIK_CONFIG

…

```
* Address ID: ...
* Body:
      - MAX_Sensors         [1 Byte]
      - MAX_Relay           [1 Byte]
      - TemperatureFormat   [1 Byte]
```

2.2.3 Program Type: (0x83) REMOTESENSOR

Request sensor values by name, e.g. outdoor, room temp/ humidity, RC switch, or get changes

table 5: Structure of a CAN Package (Program Type: REMOTESENSOR)

Address ID 29 bits	bit 0..7: Program Type	**(0x83): REMOTESENSOR**	
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*	
	bit 16..23: Function Type	// CAN: must be fix (0x00): ecsOutdor (0x01): ecsFlow (0x02): ecsCold (0x03): ecsSolar (0x04): ecsReserved1 (0x05): ecsReserved2 (0x06): ecsReserved3 (0x07): ecsStorage (0x08): ecsStorageTop (0x09): ecsStorageCenter (0x0A): ecsStorageBottom (0x0B): ecsReserved4	// room control (0x0C): ecsRcTset (0x0D): ecsRcHumidity (0x0E): ecsRcTroom (0x0F): ecsRcWheel (0x10): ecsRcSwitch
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0**	
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR	
Body 64 bits	bit 29..92: Payload	MSG_REQUEST: payload 0 Byte long MSG_RESPONSE (normal): payload 4 Bytes long data[0]: value low Byte data[1]: value high Byte data[2]: virtual circuit (Caleon)/ room controller unit (XHCC) starting with “0” data[3]: type of room controller; see “namedSensors.h” MSG_RESPONSE (°CALEON): payload 6 Bytes long data[0]: value low Byte data[1]: value high Byte data[2]: virtual circuit (Caleon)/ room controller unit (XHCC) starting with “0” data[3]: type of room controller; see “namedSensors.h” data[4]: roomIndex data[5]: roomType	

2.2.4 Program Type: (0x85) HCC

Heating circuit control program, e.g. heat request

table 6: Structure of a CAN Package (Program Type: HCC)

Address ID 29 bits	bit 0..7: Program Type	**(0x85): HCC**	
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*	
	bit 16..23: Function Type	(0): HEATREQUEST (1): HEATINGCIRCUIT_STATE1 (2): HEATINGCIRCUIT_STATE2 (3): HEATINGCIRCUIT_STATE3 (4): HEATINGCIRCUIT_STATE4	
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0**	
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR	
Body 64 bits	bit 29..92: Payload	*see individual description below*	

(0) HEATREQUEST

…

```
* Address ID: HCC.SubscriberID.HCC_HEATREQUEST.MSG_REQUEST
              HCC.SubscriberID.HCC_HEATREQUEST.MSG_RESPONSE
* Body:
      - heatRequest [1 Byte]
            The requested flow temperature (DE: VL-Temperatur) is transmitted as a
            relay value in the same way as it would be represented on a 0..10 V output
            in the "Modulate" mode. Therefore a temperature between 0 to 100 °C will be
            represented as 0 to 100 % of a total of 255, e.g. a temperature of 53 °C
            results in a value of 135. If no heat shall be requested, the value is 0.
            All controllers send their current value once per minute.

      - SolarWood   [1 Byte]
            Locally active energy source
            (0): solar load
            (1): solid fuel (wood) boiler
```

(1) HEATINGCIRCUIT_STATE1

MSG_RESPONSE
Send current heating circuit state if the state changed but not more than once every 10 seconds per circuit.

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE1.MSG_RESPONSE
* Body:
      [8 Bytes] payload
      - circuit   [1 Byte]
            circuit index
            (0)..(255)

      - state     [1 Byte]
            teRcModes from roomController.h

      - tFlowSet  [2 Bytes]
            temperature value calculated flow

      - tFlow     [2 Bytes]
            temperature value current flow

      - tStorage  [2 Bytes]
            temperature value of storage
```

MSG_REQUEST
Request heating circuit state for all circuits

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE1.MSG_REQUEST
* Body:
      [0 Byte] payload
```

(2) HEATINGCIRCUIT_STATE2
MSG_RESPONSE
Send current heating circuit state if the state changed but not more than once every 10 seconds per circuit.

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE2.MSG_RESPONSE
* Body:
      [8 Bytes] payload
      - circuit   [1 Byte]
      - wheel     [1 Byte]
            int8 -5..+5
      - tSet  [2 Bytes]
            temperature value from remote
      - tRoom  [2 Bytes]
            temperature value room
      - humidity  [2 Bytes]
            Humidity value
```

MSG_REQUEST
Request heating circuit state for all circuits

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE2.MSG_REQUEST
* Body:
      [0 Byte] payload
```

(3) HEATINGCIRCUIT_STATE3
MSG_RESPONSE
Send current heating circuit state if the state changed but not more than once every 10 seconds per circuit.

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE3.MSG_RESPONSE
* Body:
      [6 Bytes] payload
      - circuit   [1 Byte]
      - opMode     [1 Byte]
            Main operating mode
      - dewPoint  [2 Bytes]
      - pump  [1 Bytes]
            on/off
      - onReason  [1 Bytes]
            see tePumpOnReason
```

MSG_REQUEST
Request heating circuit state for all circuits

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE3.MSG_REQUEST
* Body:
      [0 Byte] payload
```

(4) HEATINGCIRCUIT_STATE4
MSG_RESPONSE

Send current heating circuit state if the state changed but not more than once every 10 seconds per circuit.

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE3.MSG_RESPONSE
* Body:
      [6 Bytes] payload
      - circuit   [1 Byte]
      - reserved   [1 Byte]
      - tMin  [2 Bytes]
      - tMax  [2 Bytes]
```

MSG_REQUEST
Request heating circuit state for all circuits

```
* Address ID: HCC.SubscriberID.HEATINGCIRCUIT_STATE3.MSG_REQUEST
* Body:
      [0 Byte] payload
```

2.2.5 Program Type: (0x8C) AVAILABLERESOURCES

table 7: Structure of a CAN Package (Program Type: AVAILABLERESOURCES)

Address ID 29 bits	bit 0..7: Program Type	**(0x8C): AVAILABLERESOURCES**	
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*	
	bit 16..23: Function Type	(0): efr_sensor (1): efr_relay (2): efr_relayID (0xFF): efr_all	
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0**	
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR	
Body 64 bits	bit 29..92: Payload	*see individual description below*	

(0) efr_sensor

MSG_REQUEST

```
* Address ID: AVAILABLERESOURCES.SubscriberID.efr_sensor.MSG_REQUEST
* Body:
      [4 Bytes] payload
      -  data[0]: ResAddr
      -  data[1]: ResBus (1wire/hts,.. -> networkTypes.h)
      -  data[2]: ResType (-> networkTypes.h)
      -  data[3]: CAN Id, which is requested
```

MSG_RESPONSE

```
* Address ID: AVAILABLERESOURCES.SubscriberID.efr_sensor.MSG_RESPONSE
* Body:
      [4 Bytes] payload
      -  data[0]: canId
      -  data[1]: ResBus
      -  data[2]: Addr
      -  data[3]: ResType
```

(1) efr_relay

MSG_RESPONSE

```
* Address ID: AVAILABLERESOURCES.SubscriberID.efr_relay.MSG_RESPONSE
* Body:
      [8 Bytes] payload
      -  data[0]: relay number 0..total-1
      -  data[1]: currently used mode
      -  data[2]: teRelayPinsFlags
      -  data[3]: instance of assigned function
      -  data[4/5]: identifier of assigned function (teFunctionId)
      -  data[6]: relay is used as second relay of this function e.g. mixer close
      -  data[7]: current relay state
```

(2) efr_relayID

MSG_RESPONSE

```
* Address ID: AVAILABLERESOURCES.SubscriberID.efr_relayID.MSG_RESPONSE
* Body:
      [8 Bytes] payload
      - String
```

2.2.6 Program Type: (0x90) PARAMETERSYNCCONFIG

table 8: Structure of a CAN Package (Program Type: PARAMETERSYNCCONFIG)

Address ID 29 bits	bit 0..7: Program Type	**(0x90): PARAMETERSYNCCONFIG**	
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*	
	bit 16..23: Function Type	(0): epsc_connect (1): epsc_factoryReset	
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0**	
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR	
Body 64 bits	bit 29..92: Payload	*see individual description below*	

(0) espc_connect

MSG_RESPONSE

```
* Address ID: PARAMETERSYNCCONFIG.SubscriberID.espc_connect.MSG_RESPONSE
* Body:
      [2 Bytes] payload
      -  data[0]: source (Configurator settings)
      -  data[1]: destination (Configurator settings)
```

(1) espc_factoryReset

MSG_RESPONSE

```
* Address ID: PARAMETERSYNCCONFIG.SubscriberID.espc_factoryReset.MSG_RESPONSE
* Body:
      [6 Bytes] payload
      -  data[0]: source (not used on receiver)
      -  data[1]: destination (if 0 than family is active)
      -  data[2]: family (if used for all devices: Caleon, CaleonBox, HCC)
      -  data[3]: synchronize (sending configurations afterwards)
      -  data[4]: disconnect (deleting configurator settings)
```

- data[5]: selftestdone (if 1, 1Wire synchro is blocked till next reboot)

2.2.7 Program Type: (0x91) ROOMSYNC

table 9: Structure of a CAN Package (Program Type: ROOMSYNC)

Address ID 29 bits	bit 0..7: Program Type	**(0x91): ROOMSYNC**
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*
	bit 16..23: Function Type	*unique CAN Subscriber ID of the receiver*
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0**
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR
Body 64 bits	bit 29..92: Payload	*see individual description below*

MSG_RESPONSE

```
* Address ID: ROOMSYNC.SubscriberID.SubscriberID.MSG_RESPONSE
* Body:
      [8 Bytes] payload

      - value   [4 Byte]
      - parameterList  [2 Byte]
      - instance of parameterList  [1 Bytes]
      - parameter ID  [1 Bytes]
```

2.2.8 Program Type: (0x94) MSGLOG

table 10: Structure of a CAN Package (Program Type: MSGLOG)

Address ID 29 bits	bit 0..7: Program Type	**(0x94): MSGLOG**	
	bit 8..15: Subscriber ID	*unique CAN Subscriber ID of the sender*	
	bit 16..23: Function Type	*tMessageSeverity*	
	bit 24..26: Protocol Type	**(0): CAN_FORMAT_0**	
	bit 27..28: CAN Message Type	(0): MSG_REQUEST (1): MSG_RESERVE (2): MSG_RESPONSE (3): MSG_ERROR	

Body 64 bits	bit 29..92: Payload	**tMessageLogCode:** *System Start* (1): MLC_SYSTEM_RESTART (2): MLC_WATCHDOG_TRIGGERED *Selftest* (20): MLC_SELFTEST_RESULT *BlueScreen* (40): MLC_BLUESCREEN_ERROR (41): MLC_BLUESCREEN_ASSAERT (42): MLC_BLUESCREEN_EXEPTION *Sensor System* (60): MLC_SENSOR_FAILURE (61): MLC_SENSOR_ISBACK (62): MLC_SENSOR_INTERNAL *Wifi* (120): MLC_WIFI_DISCONNECTED (121): MLCWIFI_RECONNECT *CAN* (140): MLC_CAN_IDSET (141): MLC_CAN_BUS_NO_CONNECTION (142): MLC_CAN_BUS_CONNECTION_RESTORED (143): MLC_CAN_OVERFLOW (144): MLC_CAN_AUTORESET_CONTROLLER_ID (145): MLC_CAN_ADD_CONTROLLER (146): MLC_CAN_DELETE_CONTROLLER_ID (147): MLC_CAN_TEST_INFO *Heating Circuit* (1000): MLC_HC_FLOWERROR (1001): MLC_HC_FLOWUNSELECTED (1002): MLC_HC_TMAXPROTECT (1003): MLC_HC_FROSTPROTECT (1004): MLC_HC_OVERLOADPROTECT (1005): MLC_HC_DISCHARGEPROTECT

MSG_RESPONSE

```
* Address ID: MSGLOG.SubscriberID.tMessageSeverity.MSG_RESPONSE
* Body:
      [8 Bytes] payload

      - param1   [4 Byte]
      - param2  [2 Byte]
      - message type (tMessageLogCode)  [2 Bytes]
```

(40): MLC_BLUESCREEN_ERROR
```
* param1: (tMessageLogParam)handlerLR
* param2: 0
```
(41): MLC_BLUESCREEN_ASSAERT
```
* param1: (tMessageLogParam)handlerLR
* param2: 0
```
(42): MLC_BLUESCREEN_EXCEPTION
```
* param1: (tMessageLogParam)frame->lr
* param2: 0
```
(62): MLC_SENSOR_INTERNAL
```
* param1: 0
* param2: LP_SENSOR_INTERNAL_HTS2XX
```
(140): MLC_CAN_IDSET
```
* param1: (currentId << 8) + id
* param2: 0
```
(141): MLC_CAN_BUS_NO_CONNECTION

* param1: 0
* param2: canIdx

(142): MLC_CAN_BUS_CONNECTION_RESTORED

* param1: 0
* param2: canIdx

(143): MLC_CAN_OVERFLOW

* param1: canOverflowCnt_log[canIdx]
* param2: canIdx

(1000): MLC_HC_FLOWERROR

* param1: vars->flowSensor.getRaw()
* param2: circuitID

(1001): MLC_HC_FLOWUNSELECTED

* param1: 0
* param2: circuitID

(1002): MLC_HC_TMAXPROTECT

* param1: vars->flowSensor.getRaw()
* param2: circuitID

2.3 CAN1, CAN2 and Routing between CAN1 and CAN2

The separation between CAN1 and CAN2 takes place on °CaleonBox. The information shared on the two bus systems are slightly different.

CAN1:

1. Outdoor temperature
2. Heating circuit information
3. Heating pump status
4. Resources

CAN2:

1. Outdoor temperature
2. Heating circuit information
3. Room information (1x worst case)

Routing CAN2 to CAN1:

1. Outdoor temperature
2. Heating circuit information
3. Heating pump status

Routing CAN1 to CAN2:

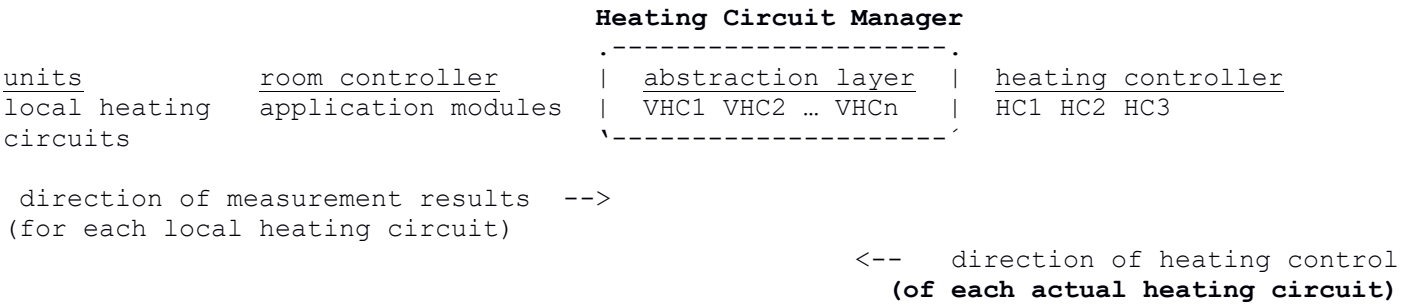
1. Outdoor temperature
2. Heating circuit information

3 Heating Circuit Manager (HCM)

3.1 Overview

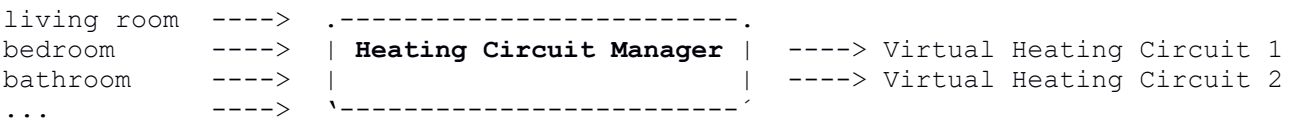
This section is a short description of the communication concept used to connect heating controllers (e.g. XHCC) with room controllers (Caleon).

The goal is to minimize the system's overall complexity while still providing enough flexibility to implement 99% of all use cases. This is done by introducing "virtual heating circuits" as an abstraction layer between the room controllers and heating controllers, i.e. the application modules and actual heating circuits.



On the input side the room controller's application modules handle individual groups of sensors/ rooms, referred to by us as "units" or "local heating circuits". In the room controller one or more such unit are assigned to a Virtual Heating Circuit (VHC). The heating controller on the other hand only sees the available virtual heating circuits and links them to actual Heating Circuits (HC). Each VHC provides an aggregated mode and temperature to the heating controller. Room controllers and heating controllers are connected via CAN.

The Heating Circuit Manager manages the assignment of each local heating circuit to a virtual heating circuit:



3.2 Influence of Room Controllers on a Heating Controller and its connected Heating Circuits

Every room controller continuously transmits Tset and Tact to the HCC. The HCC determines for each heating circuit the value pair with the largest difference. This difference is then used as the room influence (Raumeinfluss) for calculating the reference flow temperature (VL-Temp) for the respective heating circuit.

Every room controller transmits its operating mode (off, eco, regular, turbo) to the heating controller. The heating controller, however, continuous to use the operating mode derived from its own time schedule, unless every connected room controller transmits the same operating mode---only then the heating controller changes its operating mode.
Special case: Only one room controller is connected to the heating controller. In this case, this one room controller is equivalent to “all room controllers” and the heating controller works as described above.

The time schedule of the heating controller continues to exist, to be used, and to be displayed, if one or more room controllers with their own time schedule are connected.

Every time a room controller’s operating mode changes due to the programmed time schedule, the new operating mode is transmitted to the heating circuit. Note: It makes no difference, which event caused the operating mode to change---user interaction or time schedule---the effect is the same.

4 Appendix

4.1 Revision History

Date	Rev.	Description	Name
2016-05-06	1	Translate EB's document version to English	MKM
2016-05-10	2	Clarify several details	MKM
2016-05-10	3	Add description of "Heating Circuit Manager" and "REMOTESENSOR"	MKM
2016-05-11	4	Add description of "REMOTERELAY"	MKM
2016-06-17	5	Add new Function Type "HEATINGCIRCUIT_STATE" Fix order of elements in the address field description (cp. 2.1)	MKM
2016-06-20	6	Add an overview for every Program Type Fix minor inconsistencies	MKM
2016-06-20	7	Make the technical description's overview easier to understand	MKM
2016-12-06	8	Expand description of DLF_SENSOR	EB
2017-07-31	9	Add information regarding firmware updates via CAN (section 2.4). Fix inconsistencies and add details.	MKM
2017-07-31	10	Add English explanations for German constant names.	MKM
2020-08-03	11	Add AvailableResources, ParameterSyncConfig, RoomSync, MSGLog Created Customer Version	JH